COMP3011 Technical Report: EventHub API

Module: COMP3011 – Web Services and Web Data **Student:** Nathaniel Sebastian (sc232ns) **Date:** 17th March 2026

---

1. Overview

EventHub is a REST API for event management with RSVP tracking, built with FastAPI and SQLAlchemy. It integrates real-world data from Leeds City Council's Temporary Event Notices dataset (Open Government Licence v3.0) and provides analytics endpoints for event seasonality, trending detection, and personalised recommendations. The API is deployed on Render with managed PostgreSQL and supports local development using SQLite.

Repository: github.com/NathS04/comp3011-cw1-api | **Live API:** comp3011-cw1-api.onrender.com

2. Architecture

```
Client → Middleware (Rate Limit → Security Headers → ETag) → FastAPI Router → Auth → CRUD → SQLAlchemy → DB
```

Layer	Responsibility	Key Files
Middleware	Rate limiting (120/min global, 10/min login), security headers (6 headers), ETag/304, error sanitisation	`core/middleware.py`, `core/rate_limit.py`
Auth	JWT (HS256, 30-min expiry), PBKDF2-SHA256 hashing, RBAC (user/admin), event ownership	`core/auth.py`
Routes	Thin handlers; delegate to CRUD layer	`api/routes.py`, `api/analytics.py`, `api/admin.py`
CRUD	All database operations with pagination, filtering, sorting	`crud.py`
Models	6 SQLAlchemy ORM models with Alembic migrations	`models.py`
Import	XML/CSV ingestion with provenance tracking (SHA256, parser version, duration)	`scripts/import_dataset.py`

Dual database support: Analytics uses dialect-aware SQL expressions (`strftime` for SQLite, `to_char(date_trunc(...))` for PostgreSQL) via a `_month_expr()` helper, ensuring portability without schema changes.

3. Data Model and External Dataset

Six tables: `events`, `attendees`, `rsvps`, `users`, `data_sources`, `import_runs`. Events can be created manually (with `created_by_user_id` tracking ownership) or imported from external sources (with `source_id`, `source_record_id` for provenance). Attendees also track `owner_user_id`, which lets RSVP mutations be tied back to the authenticated user who created the attendee profile.

Dataset: Leeds City Council Temporary Event Notices (XML). Parsed with `defusedxml` to prevent XXE attacks. Each import run records: SHA256 hash of raw data, parser version, rows read/inserted/updated, duration, and errors. Import is idempotent—re-running updates existing records using `source_record_id` as the natural key.

Provenance endpoint: `GET /events/{id}/provenance` returns full data lineage (source name, URL, record ID, import run, parser version, SHA256 hash) for any event, distinguishing imported vs user-created events.

4. Security Model

Measure	Implementation
Authentication	JWT Bearer tokens (HS256), 30-minute expiry
Password storage	PBKDF2-SHA256 via passlib
RBAC	Three tiers: anonymous (read), authenticated (CRUD), admin (imports)
Event ownership	`created_by_user_id` on events; only owner or admin can PATCH/DELETE
Attendee / RSVP ownership	`owner_user_id` on attendees; RSVP create requires attendee owner/admin, RSVP delete allows attendee owner, event owner, or admin
Rate limiting	Sliding-window in-memory limiter; 120/min global, 10/min for `/auth/login`
Security headers	X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy, CORP, X-Request-ID
ETag caching	SHA256 body hash on GET `/events*`; `If-None-Match` returns 304 (RFC 7232)
Error sanitisation	Generic 500 message with `request_id` for log correlation; no stack traces exposed
XML safety	`defusedxml` prevents XXE attacks on imported data

5. Testing and Validation

Category	Count	Examples
Auth & JWT	9	Register, login, duplicates, expired token, invalid token
Events CRUD	9	Create, read, update, delete, pagination, filters, not-found
Ownership/Authz	17	Owner PATCH/DELETE, attendee ownership, RSVP authz, admin override
RSVPs & Stats	9	Create, duplicate rejection, not-found branches, stats
Analytics	8	Seasonality, portability, trending, recommendations
Provenance	8	Imported event, user-created, XML import, idempotency, quality endpoint
Admin/Security	7	RBAC 403, import errors, 502 propagation
Middleware	4	Headers, rate limiting, 429 headers, exception handler
ETag	3	Generation, 304, detail page
Health	2	Normal, DB error branch
Attendees	6	CRUD, not-found, events list
Headers	2	200 headers, 404 headers
Total	**84**	**95% code coverage**

Tests use in-memory SQLite with `StaticPool` for per-test isolation. Tables created and dropped per test function; runtime under 2 seconds.

6. Challenges and Trade-offs

Decision	Alternative	Rationale
In-memory rate limiting	Redis	Appropriate for single-worker free Render tier; documented as limitation
ETag via body hash	DB `updated_at` column	Simpler; no schema changes; catches list composition changes
JWT without refresh tokens	Refresh token flow	Reduces complexity for coursework scope; users re-login after 30 min
Dialect-aware analytics	PostgreSQL-only	Genuine dual-database support; tested with SQLite in CI
Event ownership (nullable FK)	Mandatory FK	Backwards-compatible with imported events that have no creator

7. Limitations

- **Rate limiting resets on restart** (in-memory; not suitable for multi-worker production)
- **No Content-Security-Policy header** (would require front-end coordination)
- **Recommendation algorithm is location-based only** (collaborative filtering would improve quality)
- **No refresh tokens** (users must re-authenticate after 30 minutes)
- **Admin promotion requires CLI script** (`scripts/make_admin.py` ; no self-service admin registration)

8. GenAI Declaration

AI tools (Gemini, Claude, ChatGPT) were used for: architecture research, middleware design comparison, ETag/RFC 7232 compliance, test generation, and debugging. All AI output was reviewed, tested, and often corrected. Key corrections: missing `requests` dependency, placeholder tests with `pass`, deprecated `Query(regex=...)`, headers not applied to 429 responses. Full logs with examples of exploring alternatives and rejecting suggestions: docs/GENAI_EXPORT_LOGS.pdf.

References

[1] Leeds City Council, "Temporary Event Notices," Data Mill North, https://datamillnorth.org/dataset/temporary-event-notices (OGL v3.0) [2] OWASP, "HTTP Security Response Headers Cheat Sheet," https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Headers_Cheat_Sheet.html [3] IETF, "RFC 7232: Conditional Requests," https://tools.ietf.org/html/rfc7232 [4] Render, "Web Services Documentation," https://render.com/docs/web-services